Name: Chaitanya Sharma
SAP ID:590026716
Batch:39
GitHub Link - https://github.com/Chaitanya189

Q1) Write a C program to traverse a singly linked list and count the number of nodes present in the list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* createNode(int value) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;
newNode->next = NULL;
return newNode;
}

int countNodes(struct Node* head) {
int count = 0;
struct Node* temp = head;
while(temp != NULL) {
count++;
temp = temp->next;
}
return count;
}

void displayList(struct Node* head) {
struct Node* temp = head;
if(temp == NULL) {
printf("List is empty\n");
return;
}
printf("Linked List: ");
while(temp != NULL) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("NULL\n");
}

int main() {
struct Node* head = NULL;
struct Node* second = NULL;
struct Node* third = NULL;
head = createNode(10);
```

```
second = createNode(20);
third = createNode(30);
head->next = second;
second->next = third;
displayList(head);
printf("Total nodes: %d\n", countNodes(head));
return 0;
}
```

Q2) Write a C program to implement a singly linked list and search for a given element in the list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* createNode(int value) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;
newNode->next = NULL;
return newNode;
}

void searchElement(struct Node* head, int key) {
struct Node* temp = head;
int position = 1;
int found = 0;
while(temp != NULL) {
if(temp->data == key) {
printf("Element %d found at position %d\n", key, position);
found = 1;
break;
}
temp = temp->next;
position++;
}
if(!found) {
printf("Element %d does not exist in the list\n", key);
}
}

void displayList(struct Node* head) {
struct Node* temp = head;
while(temp != NULL) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("NULL\n");
}
```

```
int main() {
struct Node* head = NULL;
struct Node *second, *third, *fourth;
int searchKey;
head = createNode(5);
second = createNode(10);
third = createNode(15);
fourth = createNode(20);
head->next = second;
second->next = third;
third->next = fourth;
printf("Original List: ");
displayList(head);
printf("Enter element to search: ");
scanf("%d", &searchKey);
searchElement(head, searchKey);
return 0;
}
```

Q3) Write a C program to implement insertion operations in a singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* createNode(int value) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;
newNode->next = NULL;
return newNode;
}

struct Node* insertAtBeginning(struct Node* head, int value) {
struct Node* newNode = createNode(value);
newNode->next = head;
return newNode;
}

struct Node* insertAtEnd(struct Node* head, int value) {
struct Node* newNode = createNode(value);
if(head == NULL) {
return newNode;
}
struct Node* temp = head;
while(temp->next != NULL) {
temp = temp->next;
```

```
}
temp->next = newNode;
return head;
}

struct Node* insertAfter(struct Node* head, int afterValue, int newValue) {
struct Node* temp = head;
while(temp != NULL && temp->data != afterValue) {
temp = temp->next;
}
if(temp == NULL) {
printf("Node with value %d not found\n", afterValue);
return head;
}
struct Node* newNode = createNode(newValue);
newNode->next = temp->next;
temp->next = newNode;
return head;
}

void displayList(struct Node* head) {
if(head == NULL) {
printf("List is empty\n");
return;
}
struct Node* temp = head;
while(temp != NULL) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("NULL\n");
}

int main() {
struct Node* head = NULL;
int choice, value, afterValue;
while(1) {
printf("\n--- INSERTION OPERATIONS ---\n");
printf("1. Insert at beginning\n");
printf("2. Insert at end\n");
printf("3. Insert after a node\n");
printf("4. Display list\n");
printf("5. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);
switch(choice) {
case 1:
printf("Enter value to insert at beginning: ");
scanf("%d", &value);
head = insertAtBeginning(head, value);
printf("Inserted %d at beginning\n", value);
break;
case 2:
```

```
printf("Enter value to insert at end: ");
scanf("%d", &value);
head = insertAtEnd(head, value);
printf("Inserted %d at end\n", value);
break;
case 3:
printf("Enter value after which to insert: ");
scanf("%d", &afterValue);
printf("Enter new value: ");
scanf("%d", &value);
head = insertAfter(head, afterValue, value);
break;
case 4:
printf("Current List: ");
displayList(head);
break;
case 5:
exit(0);
default:
printf("Invalid choice\n");
}
}
return 0;
}
```

Q4) Write a C program to implement deletion operations on a singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* createNode(int value) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;
newNode->next = NULL;
return newNode;
}

struct Node* deleteFromBeginning(struct Node* head) {
if(head == NULL) {
printf("List is empty\n");
return NULL;
}
struct Node* temp = head;
head = head->next;
printf("Deleted %d from beginning\n", temp->data);
free(temp);
```

```
return head;
}

struct Node* deleteFromEnd(struct Node* head) {
if(head == NULL) {
printf("List is empty\n");
return NULL;
}
if(head->next == NULL) {
printf("Deleted %d from end\n", head->data);
free(head);
return NULL;
}
struct Node* temp = head;
while(temp->next->next != NULL) {
temp = temp->next;
}
printf("Deleted %d from end\n", temp->next->data);
free(temp->next);
temp->next = NULL;
return head;
}

struct Node* deleteNode(struct Node* head, int key) {
if(head == NULL) {
printf("List is empty\n");
return NULL;
}
if(head->data == key) {
struct Node* temp = head;
head = head->next;
printf("Deleted %d\n", temp->data);
free(temp);
return head;
}
struct Node* temp = head;
while(temp->next != NULL && temp->next->data != key) {
temp = temp->next;
}
if(temp->next == NULL) {
printf("Element %d not found\n", key);
return head;
}
struct Node* toDelete = temp->next;
temp->next = toDelete->next;
printf("Deleted %d\n", toDelete->data);
free(toDelete);
return head;
}

void displayList(struct Node* head) {
if(head == NULL) {
printf("List is empty\n");
```

```
return;
}
struct Node* temp = head;
while(temp != NULL) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("NULL\n");
}

int main() {
struct Node* head = NULL;
int choice, key;
head = createNode(10);
head->next = createNode(20);
head->next->next = createNode(30);
head->next->next->next = createNode(40);
printf("Initial List: ");
displayList(head);
while(1) {
printf("\n--- DELETION OPERATIONS ---\n");
printf("1. Delete from beginning\n");
printf("2. Delete from end\n");
printf("3. Delete specific node\n");
printf("4. Display list\n");
printf("5. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);
switch(choice) {
case 1:
head = deleteFromBeginning(head);
break;
case 2:
head = deleteFromEnd(head);
break;
case 3:
printf("Enter value to delete: ");
scanf("%d", &key);
head = deleteNode(head, key);
break;
case 4:
printf("Current List: ");
displayList(head);
break;
case 5:
exit(0);
default:
printf("Invalid choice\n");
}
}
return 0;
}
```

Q5) Write a C program to reverse the data of the nodes of a singly linked list without changing the links between the nodes.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* createNode(int value) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;
newNode->next = NULL;
return newNode;
}

int getLength(struct Node* head) {
int len = 0;
struct Node* temp = head;
while(temp != NULL) {
len++;
temp = temp->next;
}
return len;
}

void reverseData(struct Node* head) {
int len = getLength(head);
if(len <= 1) return;
int* arr = (int*)malloc(len * sizeof(int));
struct Node* temp = head;
for(int i = 0; i < len; i++) {
arr[i] = temp->data;
temp = temp->next;
}
temp = head;
for(int i = len-1; i >= 0; i--) {
temp->data = arr[i];
temp = temp->next;
}
free(arr);
}

void displayList(struct Node* head) {
struct Node* temp = head;
while(temp != NULL) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("NULL\n");
```

```
}

int main() {
struct Node* head = NULL;
head = createNode(1);
head->next = createNode(2);
head->next->next = createNode(3);
head->next->next->next = createNode(4);
head->next->next->next->next = createNode(5);
printf("Original List: ");
displayList(head);
reverseData(head);
printf("After reversing data: ");
displayList(head);
return 0;
}
```

Q6) Write a C program to implement a Circular Singly Linked List and perform all basic operations on it.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* createNode(int value) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;
newNode->next = NULL;
return newNode;
}

struct Node* insertAtBeginning(struct Node* last, int value) {
struct Node* newNode = createNode(value);
if(last == NULL) {
newNode->next = newNode;
return newNode;
}
newNode->next = last->next;
last->next = newNode;
return last;
}

struct Node* insertAtEnd(struct Node* last, int value) {
struct Node* newNode = createNode(value);
if(last == NULL) {
newNode->next = newNode;
return newNode;
}
```

```
newNode->next = last->next;
last->next = newNode;
return newNode;
}

struct Node* deleteFromBeginning(struct Node* last) {
if(last == NULL) {
printf("List is empty\n");
return NULL;
}
if(last->next == last) {
printf("Deleted %d\n", last->data);
free(last);
return NULL;
}
struct Node* first = last->next;
last->next = first->next;
printf("Deleted %d from beginning\n", first->data);
free(first);
return last;
}

struct Node* deleteFromEnd(struct Node* last) {
if(last == NULL) {
printf("List is empty\n");
return NULL;
}
if(last->next == last) {
printf("Deleted %d\n", last->data);
free(last);
return NULL;
}
struct Node* temp = last->next;
while(temp->next != last) {
temp = temp->next;
}
temp->next = last->next;
printf("Deleted %d from end\n", last->data);
free(last);
return temp;
}

void displayList(struct Node* last) {
if(last == NULL) {
printf("List is empty\n");
return;
}
struct Node* temp = last->next;
printf("Circular List: ");
do {
printf("%d -> ", temp->data);
temp = temp->next;
} while(temp != last->next);
```

```
printf("(back to start)\n");
}

int countNodes(struct Node* last) {
if(last == NULL) return 0;
int count = 0;
struct Node* temp = last->next;
do {
count++;
temp = temp->next;
} while(temp != last->next);
return count;
}

int main() {
struct Node* last = NULL;
int choice, value;
while(1) {
printf("\n--- CIRCULAR LINKED LIST ---\n");
printf("1. Insert at beginning\n");
printf("2. Insert at end\n");
printf("3. Delete from beginning\n");
printf("4. Delete from end\n");
printf("5. Display list\n");
printf("6. Count nodes\n");
printf("7. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);
switch(choice) {
case 1:
printf("Enter value: ");
scanf("%d", &value);
last = insertAtBeginning(last, value);
break;
case 2:
printf("Enter value: ");
scanf("%d", &value);
last = insertAtEnd(last, value);
break;
case 3:
last = deleteFromBeginning(last);
break;
case 4:
last = deleteFromEnd(last);
break;
case 5:
displayList(last);
break;
case 6:
printf("Total nodes: %d\n", countNodes(last));
break;
case 7:
exit(0);
```

```
default:
printf("Invalid choice\n");
}
}
return 0;
}
```

Q7) Write a C program to implement a Doubly Linked List with all basic operations.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* prev;
struct Node* next;
};

struct Node* createNode(int value) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;
newNode->prev = NULL;
newNode->next = NULL;
return newNode;
}

struct Node* insertAtBeginning(struct Node* head, int value) {
struct Node* newNode = createNode(value);
if(head == NULL) {
return newNode;
}
newNode->next = head;
head->prev = newNode;
return newNode;
}

struct Node* insertAtEnd(struct Node* head, int value) {
struct Node* newNode = createNode(value);
if(head == NULL) {
return newNode;
}
struct Node* temp = head;
while(temp->next != NULL) {
temp = temp->next;
}
temp->next = newNode;
newNode->prev = temp;
return head;
}

struct Node* insertAfter(struct Node* head, int afterValue, int newValue) {
```

```
struct Node* temp = head;
while(temp != NULL && temp->data != afterValue) {
temp = temp->next;
}
if(temp == NULL) {
printf("Node with value %d not found\n", afterValue);
return head;
}
struct Node* newNode = createNode(newValue);
newNode->next = temp->next;
newNode->prev = temp;
if(temp->next != NULL) {
temp->next->prev = newNode;
}
temp->next = newNode;
return head;
}

struct Node* deleteFromBeginning(struct Node* head) {
if(head == NULL) {
printf("List is empty\n");
return NULL;
}
struct Node* temp = head;
head = head->next;
if(head != NULL) {
head->prev = NULL;
}
printf("Deleted %d from beginning\n", temp->data);
free(temp);
return head;
}

struct Node* deleteFromEnd(struct Node* head) {
if(head == NULL) {
printf("List is empty\n");
return NULL;
}
if(head->next == NULL) {
printf("Deleted %d from end\n", head->data);
free(head);
return NULL;
}
struct Node* temp = head;
while(temp->next != NULL) {
temp = temp->next;
}
temp->prev->next = NULL;
printf("Deleted %d from end\n", temp->data);
free(temp);
return head;
}
```

```
struct Node* deleteNode(struct Node* head, int key) {
if(head == NULL) {
printf("List is empty\n");
return NULL;
}
struct Node* temp = head;
while(temp != NULL && temp->data != key) {
temp = temp->next;
}
if(temp == NULL) {
printf("Element %d not found\n", key);
return head;
}
if(temp == head) {
head = temp->next;
if(head != NULL) head->prev = NULL;
} else {
temp->prev->next = temp->next;
if(temp->next != NULL) temp->next->prev = temp->prev;
}
printf("Deleted %d\n", temp->data);
free(temp);
return head;
}

void forwardDisplay(struct Node* head) {
if(head == NULL) {
printf("List is empty\n");
return;
}
struct Node* temp = head;
printf("Forward: ");
while(temp != NULL) {
printf("%d <-> ", temp->data);
temp = temp->next;
}
printf("NULL\n");
}

void backwardDisplay(struct Node* head) {
if(head == NULL) {
printf("List is empty\n");
return;
}
struct Node* temp = head;
while(temp->next != NULL) {
temp = temp->next;
}
printf("Backward: ");
while(temp != NULL) {
printf("%d <-> ", temp->data);
temp = temp->prev;
}
```

```
printf("NULL\n");
}

int countNodes(struct Node* head) {
int count = 0;
struct Node* temp = head;
while(temp != NULL) {
count++;
temp = temp->next;
}
return count;
}

int main() {
struct Node* head = NULL;
int choice, value, afterValue;
while(1) {
printf("\n--- DOUBLY LINKED LIST ---\n");
printf("1. Insert at beginning\n");
printf("2. Insert at end\n");
printf("3. Insert after a node\n");
printf("4. Delete from beginning\n");
printf("5. Delete from end\n");
printf("6. Delete specific node\n");
printf("7. Forward display\n");
printf("8. Backward display\n");
printf("9. Count nodes\n");
printf("10. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);
switch(choice) {
case 1:
printf("Enter value: ");
scanf("%d", &value);
head = insertAtBeginning(head, value);
break;
case 2:
printf("Enter value: ");
scanf("%d", &value);
head = insertAtEnd(head, value);
break;
case 3:
printf("Enter value after which to insert: ");
scanf("%d", &afterValue);
printf("Enter new value: ");
scanf("%d", &value);
head = insertAfter(head, afterValue, value);
break;
case 4:
head = deleteFromBeginning(head);
break;
case 5:
head = deleteFromEnd(head);
```

```
break;
case 6:
printf("Enter value to delete: ");
scanf("%d", &value);
head = deleteNode(head, value);
break;
case 7:
forwardDisplay(head);
break;
case 8:
backwardDisplay(head);
break;
case 9:
printf("Total nodes: %d\n", countNodes(head));
break;
case 10:
exit(0);
default:
printf("Invalid choice\n");
}
}
return 0;
}
```

Q8) Write a C program to implement a Doubly Circular Linked List.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* prev;
struct Node* next;
};

struct Node* createNode(int value) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;
newNode->prev = NULL;
newNode->next = NULL;
return newNode;
}

struct Node* insertAtBeginning(struct Node* last, int value) {
struct Node* newNode = createNode(value);
if(last == NULL) {
newNode->next = newNode;
newNode->prev = newNode;
return newNode;
}
struct Node* first = last->next;
newNode->next = first;
```

```
newNode->prev = last;
first->prev = newNode;
last->next = newNode;
return last;
}

struct Node* insertAtEnd(struct Node* last, int value) {
if(last == NULL) {
return insertAtBeginning(last, value);
}
struct Node* newNode = createNode(value);
struct Node* first = last->next;
newNode->next = first;
newNode->prev = last;
last->next = newNode;
first->prev = newNode;
return newNode;
}

struct Node* deleteFromBeginning(struct Node* last) {
if(last == NULL) {
printf("List is empty\n");
return NULL;
}
if(last->next == last) {
printf("Deleted %d\n", last->data);
free(last);
return NULL;
}
struct Node* first = last->next;
struct Node* second = first->next;
last->next = second;
second->prev = last;
printf("Deleted %d from beginning\n", first->data);
free(first);
return last;
}

struct Node* deleteFromEnd(struct Node* last) {
if(last == NULL) {
printf("List is empty\n");
return NULL;
}
if(last->next == last) {
printf("Deleted %d\n", last->data);
free(last);
return NULL;
}
struct Node* first = last->next;
struct Node* newLast = last->prev;
newLast->next = first;
first->prev = newLast;
printf("Deleted %d from end\n", last->data);
```

```
free(last);
return newLast;
}

void displayList(struct Node* last) {
if(last == NULL) {
printf("List is empty\n");
return;
}
struct Node* temp = last->next;
printf("Doubly Circular List: ");
do {
printf("%d <-> ", temp->data);
temp = temp->next;
} while(temp != last->next);
printf("(back to start)\n");
}

int countNodes(struct Node* last) {
if(last == NULL) return 0;
int count = 0;
struct Node* temp = last->next;
do {
count++;
temp = temp->next;
} while(temp != last->next);
return count;
}

int main() {
struct Node* last = NULL;
int choice, value;
while(1) {
printf("\n--- DOUBLY CIRCULAR LINKED LIST ---\n");
printf("1. Insert at beginning\n");
printf("2. Insert at end\n");
printf("3. Delete from beginning\n");
printf("4. Delete from end\n");
printf("5. Display list\n");
printf("6. Count nodes\n");
printf("7. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);
switch(choice) {
case 1:
printf("Enter value: ");
scanf("%d", &value);
last = insertAtBeginning(last, value);
break;
case 2:
printf("Enter value: ");
scanf("%d", &value);
last = insertAtEnd(last, value);
```

```
break;
case 3:
last = deleteFromBeginning(last);
break;
case 4:
last = deleteFromEnd(last);
break;
case 5:
displayList(last);
break;
case 6:
printf("Total nodes: %d\n", countNodes(last));
break;
case 7:
exit(0);
default:
printf("Invalid choice\n");
}
}
return 0;
}
```

Q9) Write a C program to represent and evaluate a Polynomial using a Linked List.

```
#include <stdio.h>
#include <stdlib.h>

struct Term {
int coeff;
int exp;
struct Term* next;
};

struct Term* createTerm(int c, int e) {
struct Term* newTerm = (struct Term*)malloc(sizeof(struct Term));
newTerm->coeff = c;
newTerm->exp = e;
newTerm->next = NULL;
return newTerm;
}

struct Term* insertTerm(struct Term* head, int coeff, int exp) {
struct Term* newTerm = createTerm(coeff, exp);
if(head == NULL || exp > head->exp) {
newTerm->next = head;
return newTerm;
}
struct Term* temp = head;
while(temp->next != NULL && temp->next->exp > exp) {
temp = temp->next;
}
```

```
newTerm->next = temp->next;
temp->next = newTerm;
return head;
}

void displayPolynomial(struct Term* head) {
if(head == NULL) {
printf("0\n");
return;
}
struct Term* temp = head;
while(temp != NULL) {
if(temp->coeff > 0 && temp != head)
printf("+");
if(temp->exp == 0)
printf("%d", temp->coeff);
else if(temp->exp == 1)
printf("%dx", temp->coeff);
else
printf("%dx^%d", temp->coeff, temp->exp);
temp = temp->next;
}
printf("\n");
}

int evaluate(struct Term* head, int x) {
int result = 0;
struct Term* temp = head;
while(temp != NULL) {
int term = temp->coeff;
for(int i = 0; i < temp->exp; i++) {
term *= x;
}
result += term;
temp = temp->next;
}
return result;
}

int main() {
struct Term* poly = NULL;
int n, coeff, exp, x;
printf("Enter number of terms: ");
scanf("%d", &n);
for(int i = 0; i < n; i++) {
printf("Enter coefficient and exponent for term %d: ", i+1);
scanf("%d %d", &coeff, &exp);
poly = insertTerm(poly, coeff, exp);
}
printf("\nPolynomial: ");
displayPolynomial(poly);
printf("Enter value of x to evaluate: ");
scanf("%d", &x);
```

```
printf("Result = %d\n", evaluate(poly, x));
return 0;
}
```

Q10) Write a C program to maintain Student Records using a Linked List.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Student {
int roll;
char name[50];
float marks;
struct Student* next;
};

struct Student* createStudent(int roll, char name[], float marks) {
struct Student* newStudent = (struct Student*)malloc(sizeof(struct Student));
newStudent->roll = roll;
strcpy(newStudent->name, name);
newStudent->marks = marks;
newStudent->next = NULL;
return newStudent;
}

struct Student* insertStudent(struct Student* head, int roll, char name[], float marks) {
struct Student* newStudent = createStudent(roll, name, marks);
if(head == NULL) {
return newStudent;
}
struct Student* temp = head;
while(temp->next != NULL) {
temp = temp->next;
}
temp->next = newStudent;
return head;
}

struct Student* deleteStudent(struct Student* head, int roll) {
if(head == NULL) {
printf("List is empty\n");
return NULL;
}
if(head->roll == roll) {
struct Student* temp = head;
head = head->next;
printf("Student with roll %d deleted\n", temp->roll);
free(temp);
return head;
}
struct Student* temp = head;
```

```
while(temp->next != NULL && temp->next->roll != roll) {
temp = temp->next;
}
if(temp->next == NULL) {
printf("Student with roll %d not found\n", roll);
return head;
}
struct Student* toDelete = temp->next;
temp->next = toDelete->next;
printf("Student with roll %d deleted\n", toDelete->roll);
free(toDelete);
return head;
}

void searchStudent(struct Student* head, int roll) {
struct Student* temp = head;
while(temp != NULL) {
if(temp->roll == roll) {
printf("Student Found: Roll: %d, Name: %s, Marks: %.2f\n", temp->roll, temp->name, temp->marks);
return;
}
temp = temp->next;
}
printf("Student with roll %d not found\n", roll);
}

void displayStudents(struct Student* head) {
if(head == NULL) {
printf("No records found\n");
return;
}
struct Student* temp = head;
printf("\n--- STUDENT RECORDS ---\n");
while(temp != NULL) {
printf("Roll: %d, Name: %s, Marks: %.2f\n", temp->roll, temp->name, temp->marks);
temp = temp->next;
}
printf("----------\n");
}

int main() {
struct Student* head = NULL;
int choice, roll;
char name[50];
float marks;
while(1) {
printf("\n--- STUDENT RECORD MANAGEMENT ---\n");
printf("1. Insert Student\n");
printf("2. Delete Student\n");
printf("3. Search Student\n");
printf("4. Display All Students\n");
printf("5. Exit\n");
printf("Enter choice: ");
```

```
scanf("%d", &choice);
switch(choice) {
case 1:
printf("Enter Roll Number: ");
scanf("%d", &roll);
printf("Enter Name: ");
scanf("%s", name);
printf("Enter Marks: ");
scanf("%f", &marks);
head = insertStudent(head, roll, name, marks);
printf("Student added successfully\n");
break;
case 2:
printf("Enter Roll Number to delete: ");
scanf("%d", &roll);
head = deleteStudent(head, roll);
break;
case 3:
printf("Enter Roll Number to search: ");
scanf("%d", &roll);
searchStudent(head, roll);
break;
case 4:
displayStudents(head);
break;
case 5:
exit(0);
default:
printf("Invalid choice\n");
}
}
return 0;
}
```